

FIAP

PROMPT ENGINEERING AND ARTIFICIAL INTELLIGENCE · 2º SEMESTRE

Memória conversacional Buffer, Summary e TokenBuffer

Aula 02/14 — Módulo 1: LangChain Foundations

 Prof. Jorge Luiz Gomes

 1h40min

 3 tipos de memória

 Andaime 45%

Agenda da Aula

1 O problema de custo da lista historico do 1º semestre

15 min

2 Os 3 tipos de memória LangChain — quando usar cada um

25 min

3 🚀 Demo ao vivo — conversa longa: Buffer vs. Summary vs. TokenBuffer

20 min

4 🖥️ Lab — chatbot do domínio do grupo com memória escolhida (45% lacunas)

25 min

5 Síntese + preview Aula 03 + CKP01 update

15 min

🎯 OBJETIVO DA AULA

Tornar a chain da Aula 01 **stateful** — com memória persistida entre turnos — escolhendo o tipo certo para o domínio do grupo, e entender o trade-off de custo de tokens de cada abordagem.

Glossário da Aula 02

Memória (Memory)

Mecanismo que permite a uma chain "lembrar" de turnos anteriores da conversa. O LLM em si não guarda nada entre chamadas — a memória é injetada de fora, como parte do prompt.

Analogia: o modelo é como alguém com amnésia a cada chamada; a memória é o caderno de anotações que alguém lê para ele antes de responder.

Stateless

Característica de APIs de LLM de não reter nenhuma informação entre requisições — cada chamada é isolada e não sabe da anterior, a menos que o histórico seja reenviado explicitamente.

ConversationBufferMemory

Tipo de memória que armazena o histórico completo da conversa, palavra por palavra, sem resumir ou descartar nada. Fidelidade máxima, custo crescente.

ConversationSummaryMemory

Tipo de memória que usa o próprio LLM para gerar, a cada turno, um resumo compacto de tudo que já foi dito — mantendo o custo de tokens estável mesmo em conversas longas.

ConversationTokenBufferMemory

Memória com janela deslizante: mantém o histórico completo até um limite de tokens (`max_token_limit`); ao ultrapassar, descarta as mensagens mais antigas.

Token

Unidade mínima de texto processada pelo modelo — fração de palavra, palavra inteira ou sinal de pontuação. É a unidade usada para medir tamanho de contexto e custo.

Context window (janela de contexto)

Quantidade máxima de tokens que o modelo consegue processar em uma única chamada — soma de prompt, histórico injetado e resposta gerada.

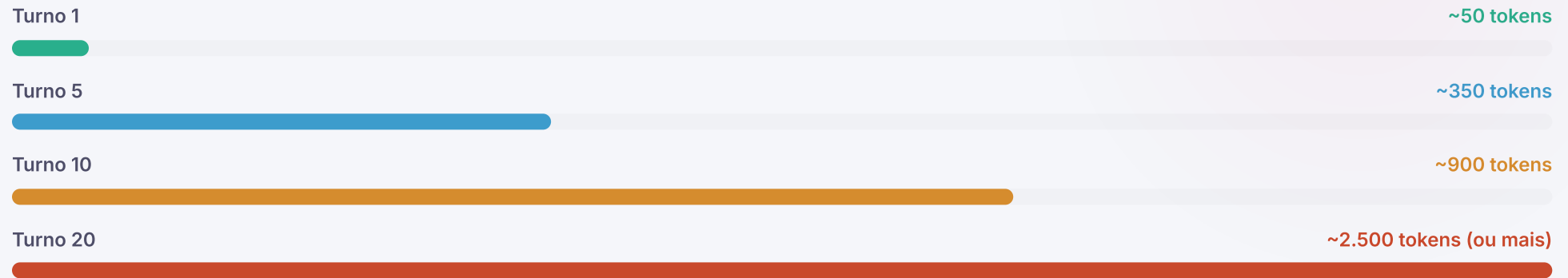
session_id

Identificador único que separa o histórico de conversa de diferentes usuários ou sessões simultâneas, usado por `RunnableWithMessageHistory` para não misturar conversas.

O problema que você já resolveu — e por que não escala

No 1º semestre, você gerenciou o histórico manualmente. Funciona — mas tem um custo crescente:

Custo de tokens a cada turno (lista historico completa reenviada)



⚠️ POR QUE ISSO É UM PROBLEMA REAL

Cada chamada ao LLM envia **todo o histórico acumulado**. Numa conversa de 50 turnos de 100 tokens cada, o turno 50 envia ~5.000 tokens só de histórico — antes mesmo de processar a nova pergunta. Em produção com múltiplos usuários simultâneos, o custo explode.

✅ A SOLUÇÃO: GERENCIAMENTO AUTOMÁTICO DE MEMÓRIA

LangChain oferece 3 estratégias de memória que resolvem o problema de formas diferentes — cada uma com trade-offs de custo, qualidade de contexto e complexidade.

01

Os 3 tipos de memória

Buffer · Summary · TokenBuffer — quando usar cada um e qual o trade-off de tokens vs. qualidade de contexto.

Os 3 tipos de memória — comparativo

BUFFER MEMORY

Tudo na janela

Guarda **todas as mensagens** integralmente. Igual à lista historico do 1º sem — mas gerenciada pelo framework.

✓ Fidelidade máxima

✗ Custo cresce

SUMMARY MEMORY

Resumo automático

Usa o LLM para **resumir o histórico** progressivamente. O contexto passa a ser um parágrafo, não uma lista de mensagens.

✓ Custo estável

⚠ Perde detalhes

TOKENBUFFER MEMORY

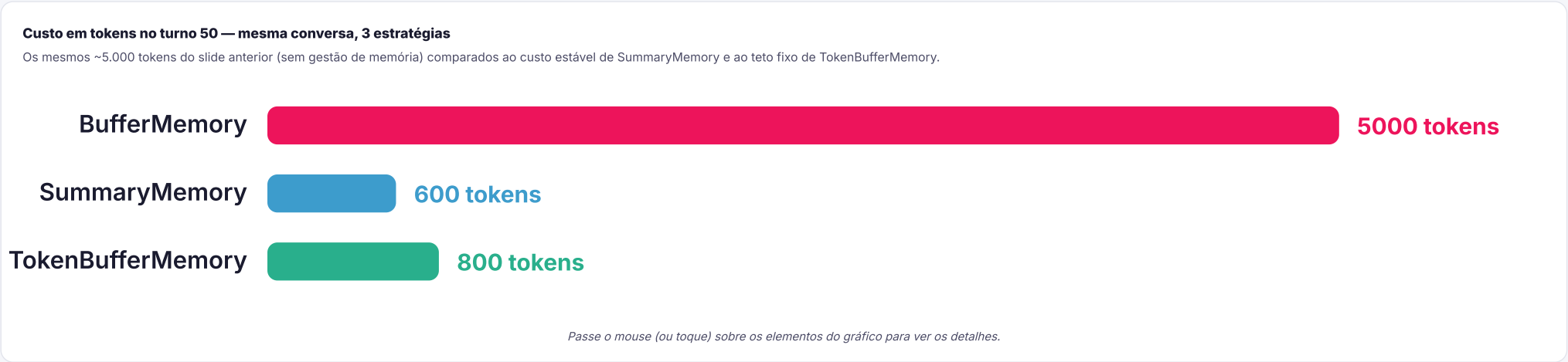
Limite por tokens

Mantém as mensagens mais **recentes** dentro de um limite de tokens configurado. Descarta o mais antigo quando o limite é atingido.

✓ Controle preciso

⚠ Perde início

CRITÉRIO	BUFFERMEMORY	SUMMARYMEMORY	TOKENBUFFERMEMORY
Custo após 50 turnos	Alto e crescente	Baixo e estável	Fixo (pelo limite)
Fidelidade do contexto	Perfeita	Aproximada	Só as últimas N
Custo de setup	Nenhum	1 call LLM extra por turno	Contagem de tokens
Melhor para	Conversas curtas, demos	Conversas longas, suporte	Controle de orçamento



FUNDAMENTAÇÃO — MEMÓRIA DE SESSÃO VS. MEMÓRIA PERSISTENTE

Freed, Jacobs e Rózsa (*Effective Conversational AI*, 2025) distinguem dois tipos de histórico em um chatbot: **session history**, o conteúdo da sessão atual — descartado quando ela termina, mas suficiente para resolver referências e perguntas de acompanhamento dentro da mesma conversa —, e **persistent user history**, que atravessa múltiplas sessões do mesmo usuário e exige esforço extra de engenharia para ser incorporada, pois permite inferir preferências ao longo do tempo. As três classes desta aula — BufferMemory, SummaryMemory e TokenBufferMemory — implementam apenas session history: vivem e morrem com a instância do `ConversationChain`. Memória persistente entre sessões exige persistir esse buffer externamente (banco de dados, vector store) — o gancho direto para RAG e ChromaDB nas Aulas 07–08.

ConversationBufferMemory — o mais simples

Equivalente direto da lista `historico` — agora gerenciada pelo framework dentro de uma `ConversationChain` :

```
from langchain_ollama import ChatOllama
from langchain_classic.memory import ConversationBufferMemory
from langchain_classic.chains import ConversationChain

llm = ChatOllama(model="gpt-oss:120b")

# Memória: guarda TUDO — crescimento linear
memoria = ConversationBufferMemory(
    memory_key="history",      # chave no contexto do prompt
    return_messages=True,      # retorna como lista de mensagens
)

# ConversationChain: gerencia prompt + memória automaticamente
chat = ConversationChain(
    llm=llm,
    memory=memoria,
    verbose=True,              # mostra o prompt completo a cada turno
)

# Conversa — o histórico é mantido automaticamente
print(chat.predict(input="Meu nome é Ana."))
print(chat.predict(input="Qual é o meu nome?")) # → "Seu nome é Ana"

# Inspeccionar o que está na memória
print(memoria.load_memory_variables({}))
# → {"history": [HumanMessage(...), AIMessage(...), ...]}
```

🔍 VERBOSE=TRUE — A FERRAMENTA DE APRENDIZADO MAIS ÚTIL

Com `verbose=True`, o LangChain imprime o prompt completo enviado ao modelo a cada turno. Use isso para entender exatamente o que está no contexto — e por que o modelo responde o que responde.

ConversationSummaryMemory — resumo progressivo

Em vez de guardar todas as mensagens, o LLM gera um **resumo corrente** que é atualizado a cada turno:

```
from langchain_classic.memory import ConversationSummaryMemory

# Summary memory usa o próprio LLM para resumir
memoria_summary = ConversationSummaryMemory(
    llm=llm,                # LLM usado para gerar o resumo
    memory_key="history",
    return_messages=True,
)

chat_summary = ConversationChain(
    llm=llm,
    memory=memoria_summary,
    verbose=True,
)

# Após vários turnos, inspecionar o resumo acumulado
chat_summary.predict(input="Meu nome é Ana e tenho 28 anos.")
chat_summary.predict(input="Trabalho como engenheira de dados.")
chat_summary.predict(input="Estou aprendendo LangChain.")

# O resumo é algo como:
# "A usuária Ana, 28 anos, engenheira de dados, está aprendendo LangChain."
# – independente de quantos turnos houve
print(memoria_summary.load_memory_variables({}))
```

⚠ CUSTO OCULTO DA SUMMARYMEMORY

A cada turno, a SummaryMemory faz **uma chamada extra ao LLM** para atualizar o resumo. Para conversas muito curtas (menos de 5 turnos), isso é mais caro que a BufferMemory. O benefício aparece em conversas longas (15+ turnos).

ConversationTokenBufferMemory — limite explícito de tokens

Mantém o histórico completo até o limite configurado, descartando as mensagens mais antigas quando o limite é atingido:

```
TOKEN_BUFFER_MEMORY.PY

from langchain_classic.memory import ConversationTokenBufferMemory

# Limite de tokens para o histórico (excluindo o novo input)
memoria_token = ConversationTokenBufferMemory(
    llm=llm,                # usado para contar tokens
    max_token_limit=500,    # máximo de tokens no histórico
    memory_key="history",
    return_messages=True,
)

chat_token = ConversationChain(
    llm=llm,
    memory=memoria_token,
    verbose=True,
)

# Com max_token_limit=500:
# - Turno 1-4: histórico completo (dentro do limite)
# - Turno 5: mensagem mais antiga é descartada automaticamente
# - Sempre mantém as mais recentes – janela deslizante

# Inspeccionar quantos tokens estão no buffer agora
historico_atual = memoria_token.load_memory_variables({})
print(f"Mensagens no buffer: {len(historico_atual['history'])}")
```

💡 ANALOGIA: JANELA DESLIZANTE

TokenBufferMemory é como uma janela deslizante: você vê sempre os últimos N tokens de conversa, e o que fica para trás da janela é esquecido. É previsível, controlável e garante que o custo nunca ultrapassa o limite definido.

ConversationChain com system prompt customizado

A `ConversationChain` usa um prompt padrão genérico. Para adicionar **persona do grupo**, injete um `PromptTemplate` customizado:

```
CHAIN_COM_PERSONA.PY

from langchain_core.prompts import PromptTemplate
from langchain_classic.chains import ConversationChain
from langchain_classic.memory import ConversationTokenBufferMemory

# Template customizado – deve conter {history} e {input}
TEMPLATE = """Você é um assistente especialista em culinária brasileira.
Seja simpático e dê dicas práticas. Sempre que der uma receita,
liste os ingredientes de forma clara.

Histórico da conversa:
{history}

Usuário: {input}
Assistente: """

prompt_custom = PromptTemplate(
    input_variables=["history", "input"], # obrigatório para ConversationChain
    template=TEMPLATE,
)

chat_culinaria = ConversationChain(
    llm=llm,
    memory=ConversationTokenBufferMemory(llm=llm, max_token_limit=800),
    prompt=prompt_custom,
    verbose=False,
)

resp = chat_culinaria.predict(input="Como faço um feijão tropeiro?")
print(resp)
```

✅ REGRA DO TEMPLATE PARA CONVERSATIONCHAIN

O template DEVE conter exatamente `{history}` e `{input}` como variáveis. O `{history}` é preenchido automaticamente pela memória; o `{input}` recebe o texto do `.predict(input=...)`.

03

Lab — Notebook Aluno

45% de lacunas — chatbot do domínio do grupo com o tipo de memória escolhido e justificativa da escolha.

Notebook Aluno — 45% de lacunas

Use o domínio escolhido na Aula 01. Complete as lacunas e justifique no comentário qual tipo de memória você escolheu e por quê.

AULA_02_2SEM_ALUNO.IPYNB

```
!pip install langchain langchain-ollama langchain-classic -q

from langchain_ollama import ChatOllama
from langchain_classic.memory import (
    ConversationBufferMemory,
    ConversationSummaryMemory,
    ConversationTokenBufferMemory,
)
from langchain_classic.chains import ConversationChain
from langchain_core.prompts import PromptTemplate
import os
from google.colab import userdata
os.environ["OLLAMA_HOST"] = "https://ollama.com"
os.environ["OLLAMA_API_KEY"] = userdata.get("OLLAMA_API_KEY")

llm = ChatOllama(model="gpt-oss:120b")

# 🚩 LACUNA 1: escreva o template com a persona do domínio do grupo
# Lembre: deve conter {history} e {input}
TEMPLATE = (____)

prompt = PromptTemplate(input_variables=["history", "input"], template=TEMPLATE)

# 🚩 LACUNA 2: escolha o tipo de memória para o domínio do grupo
# Justifique em comentário: por que este tipo e não os outros?
memoria = (____) # Buffer, Summary ou TokenBuffer

# 🚩 LACUNA 3: monte a ConversationChain com llm, memory e prompt
chat = ConversationChain((____))

# 🚩 LACUNA 4: teste com 5 turnos do domínio do grupo
for pergunta in [(____), (____), (____), (____), (____)]:
    print(f"R: {chat.predict(input=pergunta)}\n")

# Inspeccionar o estado final da memória
print(memoria.load_memory_variables({}))
```

Guia de decisão — qual memória usar no CKP01

DOMÍNIO / CENÁRIO	RECOMENDAÇÃO	POR QUÊ
Suporte técnico, atendimento ao cliente	SummaryMemory	Conversas longas, mas o resumo captura os pontos principais (problema relatado, soluções tentadas)
Tutor educacional, assistente de estudo	TokenBufferMemory	As últimas explicações são mais relevantes que as primeiras; limite de 1000–2000 tokens é suficiente
Assistente jurídico, médico	BufferMemory	Fidelidade máxima importa — perder um detalhe pode mudar o diagnóstico ou o parecer. Conversas curtas
Chatbot de receitas, entretenimento	TokenBufferMemory	Conversas curtas a médias; o usuário raramente referencia o início; controle de custo é útil
Assistente de código/programação	TokenBufferMemory com limite alto (2000–4000)	Código é verboso; o contexto recente (últimas funções discutidas) é o mais importante
Demo/apresentação curta	BufferMemory	Menos de 10 turnos — sem custo problemático, setup mais simples, funciona perfeitamente

Inspecionar e manipular a memória em código

Os 3 tipos compartilham a mesma interface — útil para debug e para o CKP01:

INSPECIONAR_MEMORIA.PY

```
# Ler o estado atual da memória (funciona em todos os tipos)
estado = memoria.load_memory_variables({})
print(estado)

# BufferMemory → {"history": [HumanMessage, AIMessage, ...]}
# SummaryMemory → {"history": "A usuária Ana é médica e..."}
# TokenBuffer → {"history": [HumanMessage, AIMessage, ...]} (últimas N)

# Adicionar memória externamente (útil para testes)
memoria.save_context(
    {"input": "Qual é a capital do Brasil?"},
    {"output": "Brasília."},
)

# Limpar completamente a memória (ex: nova sessão de usuário)
memoria.clear()
print(memoria.load_memory_variables({})) # → {"history": []}

# Acessar o buffer diretamente (Buffer e TokenBuffer)
if hasattr(memoria, "chat_memory"):
    print(f"Mensagens guardadas: {len(memoria.chat_memory.messages)}")

# Acessar o resumo (SummaryMemory)
if hasattr(memoria, "moving_summary_buffer"):
    print(f"Resumo atual: {memoria.moving_summary_buffer}")
```

Erros comuns com memória LangChain

ERRO	CAUSA	SOLUÇÃO
<code>ValueError: Missing input variables</code>	Template customizado não tem <code>{history}</code> ou <code>{input}</code>	Conferir que ambas as variáveis existem no template
Modelo "esquece" o início da conversa	TokenBuffer com limite muito baixo descartou as mensagens antigas	Aumentar <code>max_token_limit</code> ou trocar para SummaryMemory
SummaryMemory gera resumos imprecisos	O modelo de resumo (mesmo LLM) não capturou bem os detalhes	Usar um modelo menor para o resumo via <code>ConversationSummaryMemory(llm=llm_resumo)</code>
Memória não persiste entre reinicializações do kernel	Toda memória é em RAM — reiniciar o Python limpa tudo	Serializar com <code>pickle</code> ou usar <code>RedisChatMessageHistory</code> para persistência
<code>ModuleNotFoundError: langchain.memory / langchain.chains</code>	LangChain 1.0+ moveu <code>Memory</code> / <code>Chains</code> legado pra fora do pacote <code>langchain</code>	Instalar <code>langchain-classic</code> e importar de <code>langchain_classic.memory</code> / <code>langchain_classic.chains</code>

CKP01 — Aula 02 completa o requisito de memória

🚩 CKP01 — CHATBOT LANGCHAIN · ENTREGA AULA 04

- ✅ **R1 (Aula 01):** ChatOllama + ChatPromptTemplate + StrOutputParser — chain LCEL básica
- ✅ **R2 (Aula 02 — hoje):** ConversationChain com 1 tipo de memória escolhido e justificado
- **R3 (Aula 03):** PydanticOutputParser com schema Pydantic v2 — validação de saída
- **R4 (Aula 04):** Context Engineering aplicado + domínio documentado + 5 turnos

📌 O QUE DOCUMENTAR NO NOTEBOOK DO CKP01 SOBRE MEMÓRIA

1. Qual tipo escolheu e por quê (1 parágrafo). 2. Print do `load_memory_variables({})` após 5 turnos — mostra o estado da memória. 3. Se usou TokenBuffer: qual o limite configurado e por que esse valor faz sentido para o domínio.

04

Consolidação e conexões

Do laboratório aos padrões do mercado — fechando a Aula 02 com síntese, referências e o glossário técnico.

Memória conversacional na prática — padrões por tipo de produto

Produtos de IA conversacional em produção não inventam uma quarta estratégia — eles combinam os mesmos 3 padrões desta aula, escolhidos de acordo com o tipo de sessão e o custo tolerável.

CHAT DE SUPORTE AO CLIENTE

SummaryMemory na transferência de atendimento

Quando o atendimento escala de um agente de IA para um humano, o sistema gera um resumo da conversa para o novo atendente — exatamente o que a SummaryMemory faz internamente com o LLM.

ASSISTENTE DE CÓDIGO NO EDITOR

TokenBuffer com janela de código

Assistentes de IA integrados a editores mantêm os últimos N tokens de contexto de código e conversa. Quando o contexto excede o limite, o trecho mais antigo é descartado — o comportamento do TokenBufferMemory com limite configurável.

CHATBOT DE E-COMMERCE

BufferMemory com limite de sessão

Chatbots de atendimento por sessão curta (tipicamente 5–15 mensagens) mantêm o histórico completo durante o atendimento e descartam ao fechar o ticket. Para sessões curtas, Buffer é a escolha certa.

ASSISTENTE CORPORATIVO (ERP)

Memória híbrida

Assistentes embutidos em sistemas de gestão empresarial costumam combinar SummaryMemory (para capturar o que já foi configurado) com TokenBuffer (para manter os últimos comandos). A combinação dá contexto sem custo explosivo.

💡 PADRÃO, NÃO COINCIDÊNCIA

Nenhum produto real usa uma quarta estratégia mágica — a escolha é sempre um trade-off entre fidelidade, custo e tamanho típico de sessão, os mesmos 3 eixos que você aprendeu a comparar hoje.

📖 FUNDAMENTAÇÃO — 6 SINAIS DE CONTEXTO ALÉM DO HISTÓRICO

Freed, Jacobs e Rózsa (*Effective Conversational AI*, 2025, cap. 9) listam 6 tipos de informação contextual que um assistente de produção combina com a memória conversacional: **localização**, **fuso horário**, **tipo de dispositivo**, **preferências do usuário**, **padrões comportamentais** e **interações anteriores**. Este último é o único que Buffer/Summary/TokenBufferMemory cobrem — os outros 5 exigem armazenamento externo ao `ConversationChain` (perfil de usuário, sessão HTTP, banco de dados). Os autores ilustram com "Max", chatbot bancário que dá conselho genérico por ignorar que a usuária já é correntista — o mesmo risco de uma memória tecnicamente correta, mas contextualmente vazia.

Nota técnica — memória em LCEL moderno (LangChain 0.3+)

O LangChain 0.3+ prefere `RunnableWithMessageHistory` para integrar histórico em chains LCEL. `ConversationChain` ainda funciona mas é considerada legacy:

```
LCEL_COM_HISTORICO_MODERNO.PY

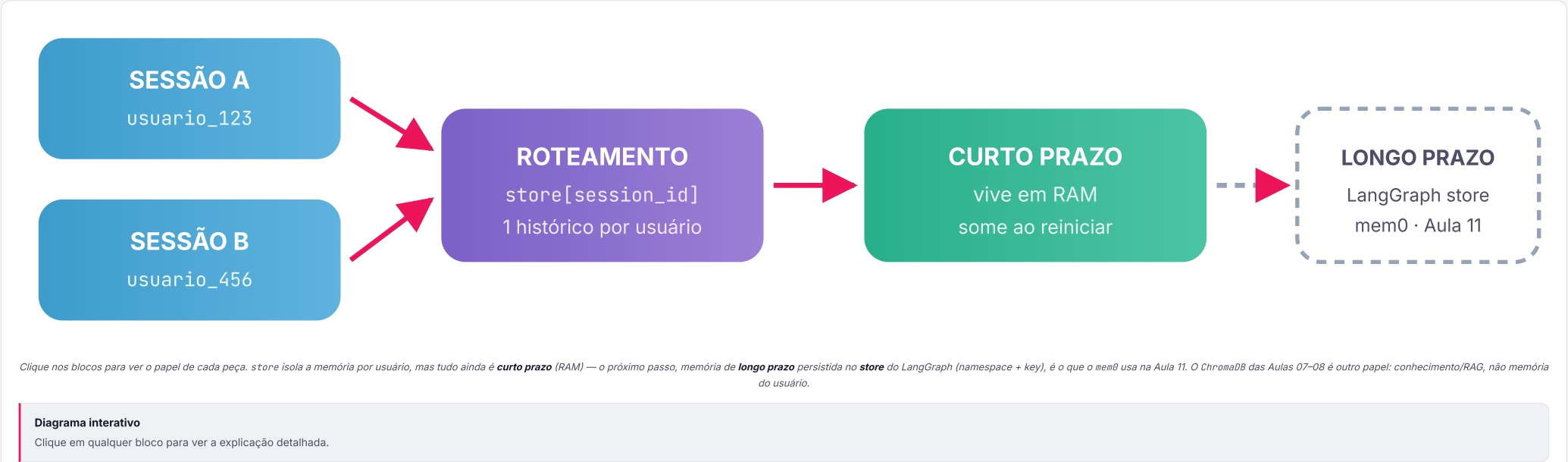
from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_core.chat_history import InMemoryChatMessageHistory

# Armazena históricos por session_id (multi-usuário)
store = {}
def get_session_history(session_id: str):
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]

# chain LCEL da Aula 01
chain_base = prompt | llm | StrOutputParser()

# Envolver com histórico — suporta múltiplas sessões simultâneas
chain_com_hist = RunnableWithMessageHistory(
    chain_base,
    get_session_history,
    input_messages_key="pergunta",
)

# Invocar com session_id — cada usuário tem seu histórico
chain_com_hist.invoke(
    {"pergunta": "Olá!"},
    config={"configurable": {"session_id": "usuario_123"}},
)
```



Síntese da Aula 02



A lista historico do 1º semestre não escala — o custo cresce linearmente com o número de turnos. Em produção com muitos usuários, isso se torna proibitivo.



BufferMemory → fidelidade máxima, custo crescente. Para demos e conversas curtas (menos de 10 turnos).



SummaryMemory → custo estável, fidelidade aproximada. Para conversas longas onde o resumo captura o essencial.



TokenBufferMemory → custo fixo, janela deslizando. Para a maioria dos casos — controle preciso com `max_token_limit`.



CKP01 — R2 entregue: ConversationChain com memória escolhida e justificada. Faltam R3 (Pydantic, Aula 03) e R4 (Context Engineering, Aula 04).

Perguntas frequentes

PERGUNTA	RESPOSTA
"Posso usar memória com a chain LCEL da Aula 01?"	Sim, com <code>RunnableWithMessageHistory</code> . Para o CKP01 use <code>ConversationChain</code> — mais simples. <code>RunnableWithMessageHistory</code> aparece na Aula 08 com Gradio.
"A memória persiste quando eu fechar o Colab?"	Não — toda memória LangChain é em RAM por padrão. Para persistência, use <code>RedisChatMessageHistory</code> ou salve com <code>pickle</code> . No CKP01, memória em RAM é suficiente.
"SummaryMemory pode resumir errado?"	Sim — o LLM pode omitir detalhes ao resumir. Por isso domínios onde cada detalhe importa (médico, jurídico) preferem Buffer ou TokenBuffer. SummaryMemory é ótima para domínios onde a essência é suficiente.
"Posso combinar dois tipos de memória?"	Sim — <code>CombinedMemory</code> permite combinar, por exemplo, Buffer para a sessão atual + Summary para histórico de sessões anteriores. Abordagem avançada que aparece no M3 (Agentes).

Python novo desta aula

CONCEITOS_PYTHON_AULA02_2SEM.PY

```
# 1. Importação de múltiplos símbolos com parênteses
from langchain_classic.memory import (
    ConversationBufferMemory,
    ConversationSummaryMemory,
)

# 2. Instância com parâmetros nomeados – keyword arguments
mem = ConversationTokenBufferMemory(
    llm=llm,
    max_token_limit=800,
    return_messages=True,
)

# 3. hasattr() – verificar se objeto tem um atributo
if hasattr(mem, "chat_memory"):
    print(mem.chat_memory.messages)

# 4. Chamada .predict() – método de conveniência da ConversationChain
resp = chat.predict(input="Minha pergunta")
# equivale a chat.invoke({"input": "Minha pergunta"})["response"]

# 5. Dict com chave computed (f-string como key não é pythônico – use variável)
estado = mem.load_memory_variables({}) # {} = nenhum input extra
historico = estado["history"]

# 6. save_context() – injetar memória manualmente (útil em testes)
mem.save_context(
    {"input": "pergunta do usuário"},
    {"output": "resposta do modelo"},
)
```

Onde estamos no semestre

✓ **A01:** chain LCEL · prompt | llm | StrOutputParser() · domínio do grupo escolhido

✓ **A02 (hoje):** ConversationChain com memória · chain stateful · CKP01 R2 entregue

○ **A03:** Pydantic v2 · PydanticOutputParser · saída com schema garantido · CKP01 R3

○ **A04:** Context Engineering · CKP01 R4 · **entrega do CKP01**

○ **A05–07:** RAG — Embeddings · ChromaDB · pipeline completo · CKP02

○ **A09–11:** Agentes · ReAct · tools · RAG como tool · Gradio · CKP03

Mais dúvidas frequentes

PERGUNTA	RESPOSTA
"Qual o max_token_limit ideal para o CKP01?"	Para a maioria dos domínios, 800–1500 tokens é um bom ponto de partida. Uma resposta média tem ~150 tokens; um turno completo (pergunta + resposta) ~200 tokens. 800 tokens mantém ~4 turnos completos com contexto confortável.
"SummaryMemory usa o mesmo modelo ou pode ser diferente?"	Pode ser diferente — você pode passar um modelo menor/mais barato para o resumo. Exemplo: <code>ConversationSummaryMemory(llm=ChatOllama(model="qwen2.5:7b"))</code> para resumir e o modelo maior para responder.
"Como mostrar no CKP01 que a memória está funcionando?"	Inclua um bloco no notebook que: (1) faz 5 turnos de conversa, (2) imprime <code>load_memory_variables({})</code> , (3) pergunta algo que depende de um turno anterior. A resposta coerente demonstra que a memória funciona.

O que vem na Aula 03 — Structured Output e Pydantic v2

⚠ O PROBLEMA QUE A AULA 03 RESOLVE

O chatbot responde em texto livre — mas quando outro sistema precisa consumir a resposta (um banco de dados, uma API, uma interface), texto livre quebra tudo.

```
resp["ingredientes"]
```

 falha se o modelo retornou
`"ingredients"` em vez de `"ingredientes"`.

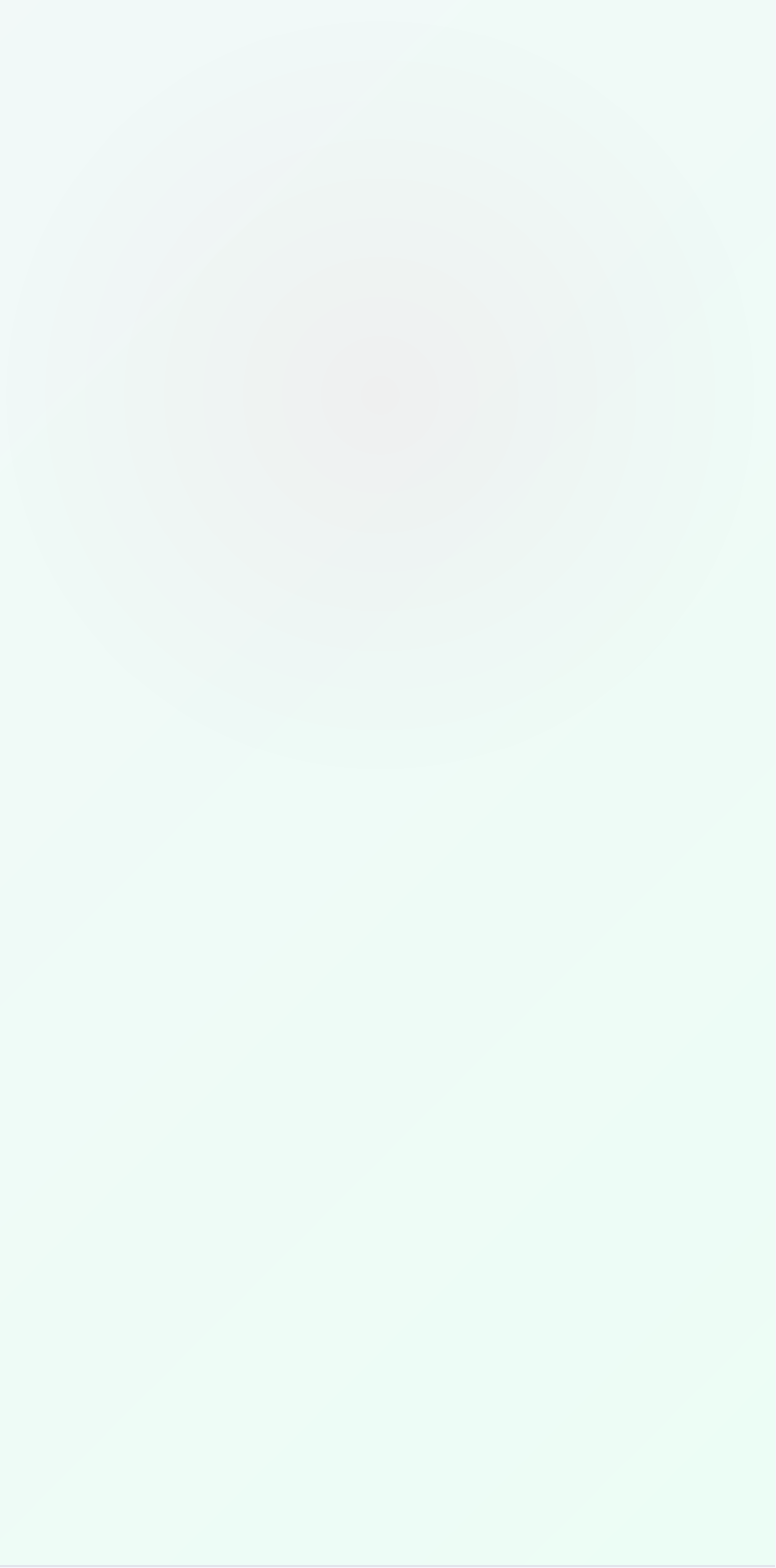
🧙 O QUE VOCÊ VAI APRENDER

Pydantic v2 + LangChain:

- `BaseModel` — definir o schema da saída
- `PydanticOutputParser` — validar automaticamente
- `ValidationError` — tratar erros de schema
- `format="json"` nativo do Ollama

O modelo **é obrigado** a retornar o schema certo — ou o parser levanta erro.

💾 **Antes da Aula 03:** mantenha o notebook de hoje com o chatbot do domínio. Na Aula 03 você vai adicionar um segundo modo de resposta: além do texto livre, o chatbot vai poder responder em JSON validado com Pydantic.



02

Demo ao vivo

Conversa longa com os 3 tipos — o aluno vê o custo crescer com Buffer e estabilizar com Summary e TokenBuffer.



Demo Prática ao Vivo

O professor executa, ao vivo no Colab, a mesma conversa de 20 turnos (perguntas progressivas de um paciente fictício) através dos 3 tipos de memória — lado a lado, com `verbose=True` mostrando o tamanho do contexto a cada turno.

BufferMemory

Contexto cresce a cada turno. No turno 20, o prompt tem ~20× mais tokens que no turno 1.

SummaryMemory

Contexto cresce nas primeiras rodadas, depois estabiliza em ~150–200 tokens de resumo, independente do número de turnos.

TokenBuffer

Contexto cresce até o limite (500 tokens), depois descarta as mensagens mais antigas. Plateau fixo.

✍️ TESTE DE MEMÓRIA AO VIVO

No turno 20, o professor pergunta: *"Qual foi a primeira coisa que te disse?"* — BufferMemory responde corretamente; TokenBuffer e SummaryMemory podem falhar, dependendo de quanto já foi descartado ou resumido.

🔧 STACK DA DEMO

Google Colab · Ollama Cloud API · `gpt-oss:120b`

Chatbot com memória — domínio do grupo ★★

Grupo 3–4 · 25 minutos · Google Colab

1. **Complete as 4 lacunas** do notebook — template com persona do domínio, tipo de memória, montagem da ConversationChain e 5 turnos de teste.
2. **Justifique a escolha** do tipo de memória em comentário no notebook. Critérios: conversas do domínio tendem a ser curtas ou longas? O usuário vai referenciar informações do início da conversa?
3. **Teste o limite da memória:** após 5 turnos, pergunte algo que dependia do turno 1. A resposta ainda é coerente?
4. **Desafio:** adicione um **segundo chat com outro tipo de memória** e compare as respostas para a mesma conversa. Qual deu respostas mais coerentes no seu domínio?

🎯 Gabarito das lacunas

Lacuna 1: f-string com persona + "Histórico:\n{history}\n\nUsuário: {input}\nAssistente:"

Lacuna 2: ConversationTokenBufferMemory(llm=llm, max_token_limit=800) — boa para a maioria dos domínios

Lacuna 3: llm=llm, memory=memoria, prompt=prompt

Lacuna 4: 5 perguntas reais do domínio escolhido na Aula 01

💡 **Dica de escolha:** domínios de suporte técnico ou jurídico → SummaryMemory (conversas longas, detalhes importam mas o resumo serve). Domínios de receitas ou jogos → TokenBuffer (conversas mais curtas, últimas mensagens são as mais relevantes).

Referências

DOCS **LangChain** — Memory: ConversationBufferMemory, SummaryMemory, TokenBufferMemory. python.langchain.com/docs/modules/memory

DOCS **LangChain** — ConversationChain: combinar memória com prompt customizado. python.langchain.com/docs/modules/chains

DOCS **LangChain** — RunnableWithMessageHistory: abordagem moderna para memória em LCEL (0.3+). python.langchain.com/docs/expression_language/how_to/message_history

DOCS **LangChain** — Short-term memory: memória dentro de uma mesma conversa/thread (checkpoint). docs.langchain.com/oss/python/langchain/short-term-memory

DOCS **LangChain** — Long-term memory: memória entre sessões (LangGraph store, namespace + key). docs.langchain.com/oss/python/langchain/long-term-memory

LIVRO **Russell, S.; Norvig, P.** — Inteligência Artificial. 3ª ed. Pearson, 2016. Cap. 3: Memória em agentes — a relevância do histórico para tomada de decisão.

PAPER **Brown, T. et al.** — "Language Models are Few-Shot Learners." NeurIPS, 2020. Seção sobre context window e por que o tamanho do contexto impacta diretamente o custo e a qualidade. arxiv.org/abs/2005.14165

EBOOK **Freed, A.; Jacobs, C.; Rózsa, E.** — Effective Conversational AI. Manning, 2025. Cap. 9: Harnessing Context for an Adaptive Virtual Assistant Experience — a distinção entre session history e persistent user history que fundamenta esta aula.

FIAP

Aula 02 concluída 🧠

A chain da Aula 01 agora tem memória. Na Aula 03 ela ganha saída estruturada com Pydantic — e o CKP01 fica quase completo.

→ PRÓXIMA AULA — AULA 03 · 17/08

Structured output e Pydantic v2

Forçar o LLM a responder com um schema garantido. CKP01 R3 entregue.

📌 CKP01 progresso

R1 ✅ · R2 ✅ · R3 (Aula 03) · R4 (Aula 04). Entrega na Aula 04.